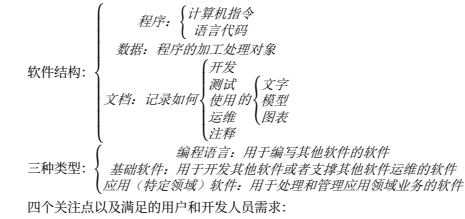


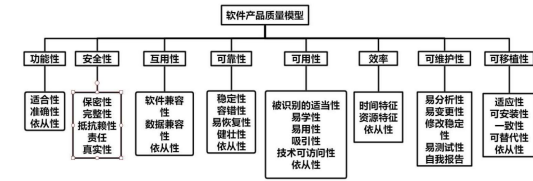
1. 软件：一系列按特定顺序组织的计算机指令和数据的集合



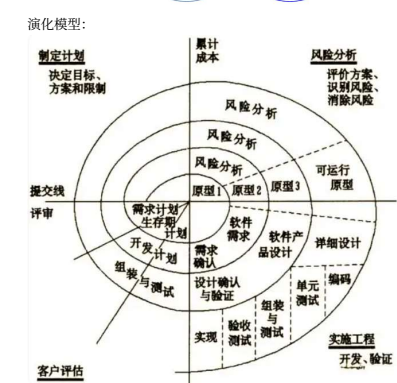
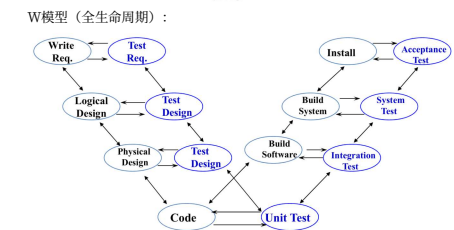
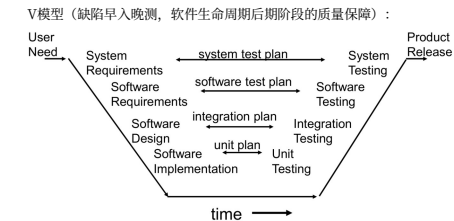
四个关注点以及满足的用户和开发人员需求：
软件产品能力 越强越好
软件产品质量 越高越好
软件开发和运维成本 越低越好
软件开发效率 越快越好

2. 软件质量：软件与明确或隐含定义的需求相一致的程度

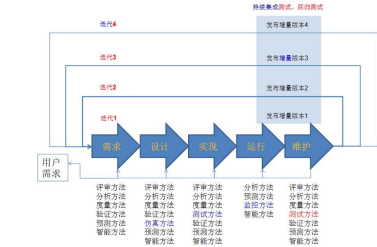
具体内容：{ 明确定义：明确叙述的功能和性能需求、文档中明确描述的开发标准
隐含定义：任何专业开发的软件产品都应该具有的隐含特征 }
外延：{ 软件的各种质量, 各种软件的质量 }
软件质量评估模型/标准：ISO/IEC 25010



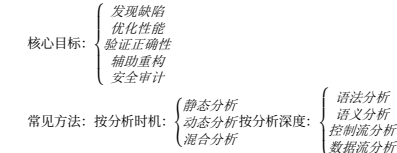
3. 软件全生命周期质量保障：单个生命周期的每个阶段、多生命周期中每个生命周期每个阶段的质量保障



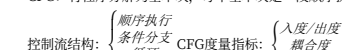
5. 全方位软件缺陷检测（监控针对外部质量，其余内部）



7. 程序分析：通过系统化方法研究程序行为、结构、性能或安全性，旨在理解、验证或优化软件。通过自动化或半自动化工具，帮助开发者发现代码缺陷、提升效率、保障安全，并辅助复杂系统的维护。编译器设计、软件工程和网络安全领域的重要基础。



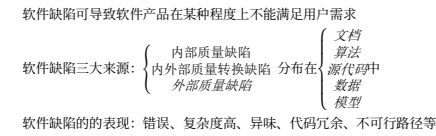
控制流分析：用于研究程序代码的执行顺序和逻辑路径。通过分析程序中的控制结构，推断程序运行时的可能行为，从而支持优化、错误检测或安全验证等任务。
CFG：将程序分解为基本块，每个基本块是一段顺序执行的代码（无跳转）；有向边表示基本块之间的跳转关系



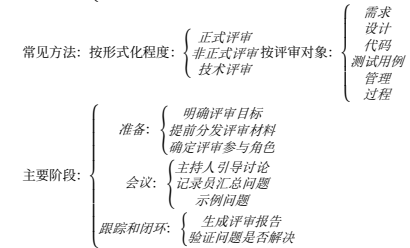
4. 全生命周期软件质量的保证：{ 先验方法：软件开发管理, 后验方法：软件缺陷/错误检测 }



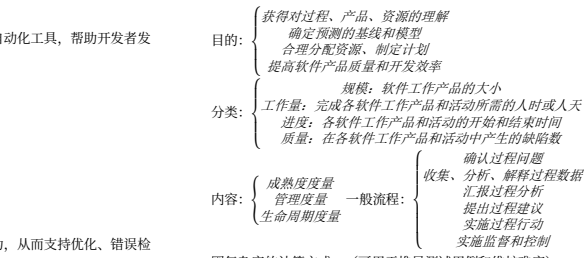
软件缺陷：
（1）计算机软件或程序中存在的某种破坏正常运行能力的问题、错误，或者隐藏的功能缺陷。
（2）产品内部看：软件产品开发或维护过程中存在的错误、毛病等各种问题。产品外部看：使得系统需要实现的某种功能的失效或违背，可导致软件发生故障。
软件缺陷可导致软件产品在某种程度上不能满足用户需求



6. 软件评审：一种系统化的质量保障活动，通过团队协作对软件相关文档、设计、代码或流程进行检查。目的是尽早发现缺陷、优化实现方案、确保交付成果符合需求/质量标准。是预防缺陷扩散和降低开发风险的核心实践。



8. 软件度量：对软件开发项目、过程和产品进行数据定义、收集和分析的持续性量化过程。目的是理解、预测、评估、控制和改善软件开发过程和产品。通过软件度量可以改进开发过程，促进项目成功，开发高质量的软件产品。



圈复杂度的计算方式：（可用于推导测试用例和维护难度）
（1）边-节点+2
（2）CFG中区域数量
（3）决策节点数+1

测或安全验证等任务。

CFG：将程序分解为基本块，每个基本块是一段顺序执行的代码（无跳转）；有向边表示基本块之间的跳转关系

控制流结构： $\begin{cases} \text{顺序执行} \\ \text{条件分支} \\ \text{循环} \\ \text{函数调用} \end{cases}$ CFG度量指标： $\begin{cases} \text{入度/出度} \\ \text{耦合度} \\ \text{圈复杂度} \end{cases}$

数据分析：用于追踪程序中数据的定义、使用和传播过程，分析变量或表达式在程序执行时的可能状态（如值、生命周期、依赖关系）。通过动静态方法，推导数据在程序中的流动路径，从而支持代码优化，错误检测和安全验证。

作用： $\begin{cases} \text{代码优化} \\ \text{检测错误} \\ \text{安全分析} \\ \text{并行化支持} \end{cases}$

DFG：将程序表示为数据流动的节点和边，节点对应程序操作，边表示数据依赖关系。

属性： $\begin{cases} \text{到达定义} \\ \text{活跃变量} \\ \text{可用表达式} \end{cases}$

传递函数：描述程序操作如何影响数据流状态

对于结构化程序，数据流分析发生在AST上，跨过程（全局）数据流分析发生在CFG上。且大多数问题难求精确解。

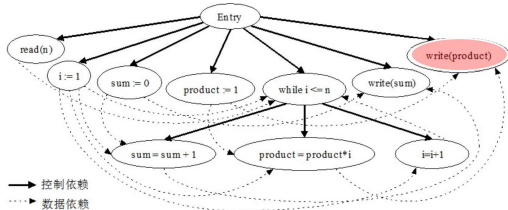
数据流分析是一种近似分析，其可以高估解，这样的系统不遗漏信息，因此更安全、保守，适用于优化。也可以低估解，其可能遗漏信息，因此不安全，适用于软件工程任务。数据流分析的最大挑战在于高效地提供安全而精确（最小化额外信息）的信息。

	解释	定义
到达定义	对基本块s寻找程序中到达s边界的所有定义	程序点p到程序点q是在某条路径上是可达的当且仅当这个定义在该路径上不会被重定义
活跃分析	变量若将在未来适用，则其在程序某点活跃。所以活跃性分析需要考虑未来	变量在程序点P处活跃当且仅当存在一条从P出发的路径，该路径中变量在重新定义之前被使用。

程序切片：给定某个程序的兴趣点和兴趣变量（或变量集合）(S,V)，观察程序中哪些语句对S处的变量V有影响；或者观察S处的变量V对程序中的哪些语句有影响。

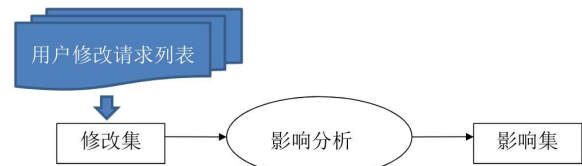
计算步骤：

- 构造例子程序的程序依赖图
- 图可达性算法：找出所有从依赖图入口节点到切片准则节点的路径，标记路径经过的节点从而构成程序关于切片准则的（过程内）依赖图切片，这些节点对应的源程序的语句和控制谓词构成的集合就是该程序关于切片准则的程序切片。

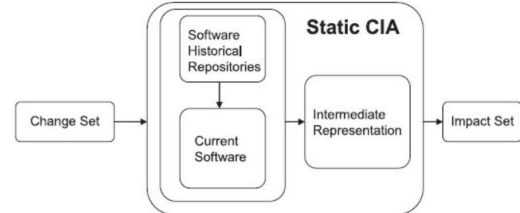


修改影响分析：

原因：对软件修改时候会对其他部分造成潜在影响，从而导致软件不一致性，若实施修改成本高、影响范围广，甚至超过重新开发该软件的成本，就需要考虑其他替代方案或者重新开发软件。若接受修改，我们需要准确预测修改带来的波动效应。其既可以提高维护人员实施修改的信心，可以帮助维护人员准确找到需要二次修改的代码，节省维护时间。



静态方法：考虑所有输入和可能性为，高成本保守估计



动态方法：考虑特定输入和运行时收集的数据（执行跟踪、执行关系、覆盖）来计算影响集
动态程序分析是基于程序执行的分析技术，收集程序的执行信息不可或缺。方法包括：

- 程序追踪：完整的记录程序执行中的控制流、数据流，以便再现该次执行
- 路径剖析：仅收集路径信息，耗费较小、侧重于路径的执行频率
- 边的剖析：记录控制流中每条边的执行频率

路径剖析的步骤： $\begin{cases} \text{输入：待路径剖析程序+CFG} \\ \text{路径剖析算法：根据CFG得出需要在程序中插装的探针语句及位置} \\ \text{插装：将算法得出的探针语句插装到程序相应的位置中} \\ \text{执行：执行插装后的程序若干次，并收集路径编码} \\ \text{输出：根据收集的信息统计所执行的路径及其相应的频率} \end{cases}$

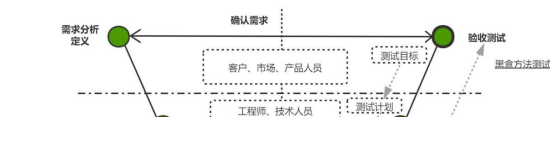
PAP方法：在单变量剖析方法的基本过程上，改进探针插装算法，设计了回溯算法以通过探针值获取及相应的路径；提供断点机制处理探针值溢出的情况，并在CFG的可规约无环子图中结合使用EPP方法进行剖析，改进了效率。

- 软件测试：用于评估软件产品质量，旨在通过执行软件的各个功能、检查程序行为等操作，发现软件中的缺陷、错误或者不符合规格需求说明书的地方。目的是确保软件产品能够满足用户需求、具有较高的质量和可靠性。

意义：是软件工程的重要组成部分，软件质量保证的重要前提，软件测试贯穿软件开发生命周期，通过验证和确认活动过程保证质量
目标：以最少的时间和人力，尽早尽可能多地发现程序中的错误和缺陷，并证明软件的功能和性能与需求说明相符合。

原则： $\begin{cases} \text{尽早不断地测试，问题发现越早，解决代价越小} \\ \text{应追溯到用户需求，从小到大规模，并对每个测试结果全面检查} \end{cases}$
两种定义：（软件缺陷发现的主要手段）

- 通过在某种特定输入下运行软件，观察实际输出结果是否满足预期结果的一种技术手段
- 在规定条件下对程序进行操作，以发现程序错误，衡量软件质量，并对其是否能满足设计要求进行评估



图复杂度的计算方式：（可用于推导测试用例和维护难度）

- 边-节点+2
- CFG中区域数量
- 决策节点数+1

- 形式化验证：使用数学工具分析设计可能行为空间。通过数学建模和严格的推理规则来验证设计的正确性，确保设计在所有可能的输入下都能满足预期的功能和性质。

基本原理：使用数学工具对设计进行建模，通过穷举系统运行过程中电路能达到的所有状态，以断言的形式完成设计的功能验证和规则检查。

应用场景：适用于综合前后的等价性检查和RTL设计的功能验证，可通过完全的覆盖率并能够发现真实难以发现的极端情况。

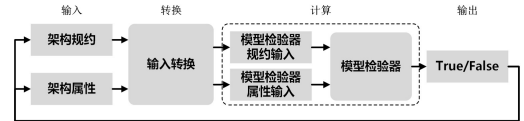
优势： $\begin{cases} \text{完全覆盖} \\ \text{精确性和可靠性} \\ \text{自动化} \end{cases}$ 局限性： $\begin{cases} \text{计算复杂度} \\ \text{依赖强大的运算系统} \end{cases}$

三种类型：

- 定理证明：将模型抽象为逻辑公式，然后使用自动的逻辑推理技术验证是否正确。一般使用HOL进行证明
- 模型检验：检查RTL代码是否满足规范中规定的一些特性
- 等效性检验：验证一个设计经过变换后，穷尽地检验变化前后的功能一致性。

实质：以数学理论为基础，用一套特定的符号和技术对软件系统进行描述和分析，以期提高软件可靠性。

研究意义： $\begin{cases} \text{提供描述手段，精确、无歧义地描述系统赋予程序的意义} \\ \text{提供分析手段以证明系统正确性} \\ \text{帮助开发人员找出系统出错原因提供开发方法和工具} \\ \text{实现系统开发过程中全部或部分开发活动的(半)自动化} \end{cases}$



模型检验SA的一般过程

- 软件模拟：一种用于模拟真实环境中软件运行过程的工具，通过建立数学模型，模拟软件的行为或运行情况，开发者可以在软件投入实际运行之前，推测其在各种条件下的表现，检测软件的稳定性、性能以及是否存在潜在问题。

工作原理：通过建立数学模型来模拟真实环境中的软件运行过程。这种

模拟可以是 $\begin{cases} \text{系统的} \\ \text{网络的} \\ \text{特定功能的} \end{cases}$ 。通过模拟，开发者可以在不同环境和条件下

测试软件，确保软件的适应性和稳定性。

应用场景：如操作系统、数据库管理、网络通信。在复杂/大型软件开发中，可以帮助开发者创建一个可控的测试环境，以便更好地了解软件的运行情况并优化。此外，模拟软件课可以用来预测和测试软硬件之间的兼容性。

重要性： $\begin{cases} \text{帮助开发者提前发现和解决潜在问题} \\ \text{加速软件的开发周期} \\ \text{提高软件的质量和用户体验} \end{cases}$ 。通过模拟，让开发者

可以在不同的环境和条件下测试软件，确保软件的适应性和稳定性。

基于仿真模拟的软件缺陷检测方法：在虚拟环境中模拟真实运行场景，动态执行软件并观察行为，从而发现潜在缺陷。适用于复杂系统或高环境依赖的缺陷检测。

仿真软件是通过数学模型和计算机算法，模拟和重现真实系统的行为和性能的软件工具。对系统进行多方位、多层次的模拟和分析，以提供全面、准确的系统行为预测。

类型： $\begin{cases} \text{架构模拟} \\ \text{系统模拟} \end{cases}$

架构模拟的五个阶段： $\begin{cases} \text{架构建模} \\ \text{仿真配置} \\ \text{仿真产生} \\ \text{配置评价} \\ \text{结果评估} \end{cases}$

- 软件运行监控：通过专门的软件工具或系统，对软件应用程序的运行状况进行实时监测和控制，以确保其性能、可用性和安全性。通常包括对软件运行状态的实时跟踪、性能指标的记录和分析、错误检测和异常处理。

作用：确保软件稳定运行、及时发现并解决问题，从而提高系统的可靠

性和用户体验，具体包括： $\begin{cases} \text{性能监测} \\ \text{可用性监测} \\ \text{错误检测与处理} \\ \text{安全监控} \end{cases}$

实现方法与技术手段： $\begin{cases} \text{日志分析} \\ \text{性能监控工具} \\ \text{错误处理机制} \\ \text{安全监控系统} \end{cases}$

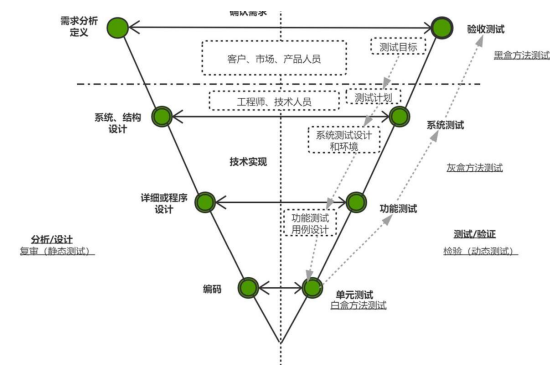
传统的静态验证的方法已不适用于运行时验证WEB服务组合的正确性，需要采用运行时监控技术作为静态验证技术的有益补充，来动态地验证WEB服务组合过程的执行轨迹是否与规定的属性一致

- 基于知识的方法：利用知识图谱描述领域知识及关系，获得领域知识图谱，再构建问题知识图谱，从而检测一致性。支持主流程程序设计语言

- 智能化方法：利用机器学习的各种模型

智能软件工程： $\begin{cases} \text{智能化开发技术} \\ \text{智能化管理技术} \\ \text{智能化运维技术} \end{cases}$

- 软件缺陷预测：一种通过分析软件代码、开发过程数据或历史信息，提前识别软件中可能存在的缺陷或者风险模块的技术。核心目标是帮助开发团队优化测试资源分配、提高软件质量，并降低修复成本与缺陷检测的区别：



16. 软件缺陷预测：一种通过分析软件代码、开发过程数据或历史信息，提前识别软件中可能存在的缺陷或者风险模块的技术。核心目标是帮助开发团队优化测试资源分配、提高软件质量，并降低修复成本与缺陷检测的区别：

- (1) 缺陷检测：在代码中直接发现已存在的错误
- (2) 缺陷预测：在缺陷实际出现之前，预测哪些代码模块或组件未来可能发现问题

预测对象：{ 模块级
代码变更级
版本级
版本内预测
跨版本预测
跨项目预测
应用范围：{ 跨组织/企业预测
跨领域/行业预测
跨国家/地区预测

